

torch_vn_algebra: batched random operators, states, channels and constrained optimisation in finite-dimensional Type I von Neumann algebras

Irina Nikolaeva

Andrej Novikov

Abstract

`torch_vn_algebra` is a PyTorch library for numerical work in algebras $\mathcal{M} = \bigoplus_{c=1}^C M_{k_c}(\mathbb{C})$, the finite-dimensional Type I von Neumann algebras that describe quantum systems with superselection rules. Operators, states and channels are stored as batched, block-diagonal tensors, so that thousands of random samples, or thousands of independent starts of an optimisation, are processed at once on a CPU or a GPU. The library provides random operators with prescribed spectra and Haar-distributed eigenvectors, the three trace functionals that are natural on $\mathcal{M}$, states and Lüders conditioning, completely positive maps within and between sectors, tensor products and partial traces, Hamiltonian and Lindblad dynamics, differentiable parametrisations with a prescribed Michelson contrast, and safeguards that estimate run time and memory before expensive steps. We describe the design, show how the library is used on short complete examples, validate it against a set of exact results from random matrix theory and quantum information, and apply it to the trace inequalities studied in our earlier work. There, constrained optimisation replaces Monte Carlo sampling: it recovers the sharp bounds that sampling misses in all but the smallest dimensions.

1 Introduction

Block-diagonal operators appear whenever a symmetry or a superselection rule splits the Hilbert space into sectors: charge or particle-number sectors, parity, total spin, or the classical part of a hybrid classical–quantum system. Mathematically, the observables form a finite-dimensional Type I von Neumann algebra $\mathcal{M} = \bigoplus_c M_{k_c}(\mathbb{C})$, whose centre is spanned by the sector projections, and the choice of trace on $\mathcal{M}$ is no longer unique. The questions we had in mind when writing this library come from operator theory: which inequalities between traces of products characterise the trace or the centre of $\mathcal{M}$ [2, 3, 4, 5], and how far do they fail in terms of a spectral quantity such as the Michelson contrast [6, 7]?

General-purpose quantum software (QuTiP [8], Qiskit [9], the JAX-based `dynamics` [10]) is built around a single Hilbert space; sectors, several traces and large batches of random operators with prescribed spectra are not first-class objects there. The niche of `torch_vn_algebra` is this combination: the algebra $\mathcal{M}$ and its traces as the basic objects, batched random sampling and batched optimisation over them, and enough quantum information and dynamics (states, channels, tensor products, Lindblad evolution) to formulate physical questions. We do not aim to replace the packages above for single-system simulations.

This text supersedes the description of the first version of the library [1]; the list of corrections to that version is part of the repository.

Contributions. (i) A batched block-diagonal data model with three trace conventions and explicit conversions between them (Sections 2, 3). (ii) States, channels within and between sectors, tensor

products and dynamics that respect the sector structure (Section 4). (iii) A validation suite against exact results (Section 5). (iv) Differentiable parametrisations with a hard Michelson-contrast constraint and batched multistart optimisation, applied to sharp bounds of trace inequalities (Section 6). (v) Run-time and memory safeguards (Section 3).

2 Mathematical setting

Algebra and traces. Let $\mathcal{M} = \bigoplus_{c=1}^C M_{k_c}(\mathbb{C})$ act on $\mathcal{H} = \bigoplus_c \mathbb{C}^{k_c}$; $A \in \mathcal{M}$ is block diagonal, $A = \bigoplus_c A_c$. Every trace on $\mathcal{M}$ is $\text{tr}_w(A) = \sum_c w_c \text{Tr } A_c$ with weights $w_c > 0$. The library implements

$$\text{Tr}_{\text{blunt}} : w_c = 1, \quad \text{Tr}_{\text{norm}} : w_c = \frac{1}{k_c}, \quad \tau_{\text{vN}} : w_c = \frac{1}{Ck_c},$$

where τ_{vN} is the faithful normal tracial state that gives every sector the same weight. For $A \in \mathcal{M}$, $|A| = (A^*A)^{1/2}$; for $A \geq 0$ the Michelson contrast is $\Delta(A) = (\lambda_{\max} - \lambda_{\min})/(\lambda_{\max} + \lambda_{\min}) \in [0, 1]$, with the extreme eigenvalues taken over all sectors; $\Delta(A) = 0$ if and only if A is a multiple of the identity.

States. A state ω on $\mathcal{M}$ is represented by its density $\rho \geq 0$ with respect to Tr_{blunt} , $\omega(A) = \text{Tr}(\rho A)$, $\sum_c \text{Tr } \rho_c = 1$. The restriction of ω to the centre is the classical distribution $p_c = \text{Tr } \rho_c$ over sectors. The density of the same state with respect to tr_w is $\rho_c^{(w)} = \rho_c/w_c$; the library converts between the conventions explicitly, because statements such as “trace preserving” or “the dual map” depend on them as soon as the sectors change.

Maps between sectors. Let $\mathcal{M}_{\text{in}} = \bigoplus_c M_{k_c}$ and $\mathcal{M}_{\text{out}} = \bigoplus_d M_{m_d}$. Every normal completely positive map $\Phi : \mathcal{M}_{\text{in}} \rightarrow \mathcal{M}_{\text{out}}$ (in the Schrödinger picture) is a sum of completely positive maps between blocks,

$$\Phi(\rho)_d = \sum_c \sum_i K_i^{dc} \rho_c (K_i^{dc})^*, \quad K_i^{dc} : \mathbb{C}^{k_c} \rightarrow \mathbb{C}^{m_d}.$$

It preserves Tr_{blunt} iff $\sum_{d,i} (K_i^{dc})^* K_i^{dc} = 1_c$ for all c , and $T_{dc} = \text{Tr } \Phi_{dc}(\rho_c)/\text{Tr } \rho_c$ is the classical transition matrix between sectors. The dual with respect to tr_w on both sides is $\Phi^{*,w}(A)_c = \sum_{d,i} (w_d^{\text{out}}/w_c^{\text{in}}) (K_i^{dc})^* A_d K_i^{dc}$; for sector-preserving maps on one algebra the weights cancel and all three traces give the same dual.

Tensor products. $(\bigoplus_c M_{k_c}) \otimes (\bigoplus_d M_{m_d}) = \bigoplus_{(c,d)} M_{k_c m_d}$ is again of Type I with sectors labelled by pairs. The partial trace $(\text{Tr}_2 X)_c = \sum_d \text{Tr}_{m_d} X_{(c,d)}$ is a map between sectors in the sense above; its dual with respect to Tr_{blunt} is the embedding $A \mapsto A \otimes 1$.

Dynamics. For $H = H^* \in \mathcal{M}$ and jump operators $L_j \in \mathcal{M}$ the Gorini–Kossakowski–Sudarshan–Lindblad equation $\dot{\rho} = -i[H, \rho] + \sum_j \gamma_j (L_j \rho L_j^* - \frac{1}{2}\{L_j^* L_j, \rho\})$ preserves every sector, so the sector probabilities p_c are conserved (superselection).

3 Design

Data layout. An operator is a tensor of shape $(B, C, k_{\max}, k_{\max})$: B samples, C sectors, block c in the top-left $k_c \times k_c$ corner and zeros elsewhere. Linear algebra is applied to the batch of active blocks of each sector, so one call to a batched LAPACK/cuSOLVER routine serves all samples. The padding costs memory when the sector sizes differ strongly; a bucketed layout is planned (Section 8).

Objects. `Operator` (with `DensityMatrix` as a subclass for states), `Channel` (sector preserving) and `InterSectorChannel` (between sectors or algebras) are thin wrappers around such tensors; Table 1 lists the modules. Operations on operators are lazy: they record how to compute the result and do so on first use; the result is cached and the recipe dropped, so intermediates of long expressions can be freed. Laziness defers and skips work but does not fuse operations; where this matters the library uses direct formulas, e.g. $\text{Tr}(\rho A) = \sum_{ij} \rho_{ij} A_{ji}$ in $O(k^2)$ instead of forming ρA .

module	content
<code>algebra</code>	<code>TypeIAlgebra</code> , <code>Operator</code> : random operators, unitary ensembles, traces, functional calculus
<code>states</code>	<code>DensityMatrix</code> , random states, Gibbs states, Lüders rule, entropies, fidelity
<code>channels</code>	<code>Channel</code> , <code>InterSectorChannel</code> , Choi matrices, standard and random channels
<code>composite</code>	tensor products, <code>kron</code> , partial traces
<code>dynamics</code>	exact Schrödinger/von Neumann evolution, RK4, Lindblad equation, $e^{t\mathcal{L}}$
<code>optimize</code>	parametrisations with prescribed contrast, batched multistart <code>extremize</code>
<code>cost</code>	run-time and memory estimates, warnings and errors

Table 1: Modules of `torch_vn_algebra`.

Random unitaries. Haar unitaries on $U(n)$ and $O(n)$ are obtained from the QR decomposition of a Ginibre matrix with i.i.d. circular Gaussian entries after fixing the phases of $\text{diag } R$ [11]; $SU(n)$ and $SO(n)$ by multiplying with $\det^{-1/n}$ or by a sign flip of one column; the circular orthogonal and symplectic ensembles as $W^T W$ and $W^R W$ with W Haar on $U(n)$ [12]. Section 5 checks all of them.

Precision and differentiation. Algebras are created in single (default) or double precision. All operations are composed of differentiable PyTorch primitives, so gradients with respect to the matrix entries are available; eigenvalue and singular-value decompositions have singular gradients at degenerate spectra, which the parametrisations of Section 6 avoid.

Safeguards. Batched computations make it easy to request hours of work or more memory than the device has. The `cost` module therefore (a) times the first few items of every large batched eigen-, singular-value, QR or exponential decomposition and issues a warning with the expected total time when it exceeds a threshold (10 s by default), (b) compares the estimated peak memory of large allocations with the free memory of the device, warning above 50 % and raising an `InsufficientMemoryError` above 90 %, and (c) reports the expected duration of time-stepping and optimisation loops after their second step, optionally with a progress bar. All thresholds can be changed, and the checks can be switched off locally.

4 Usage by example

The listings of this section are complete programs from `examples/paper`; the test suite runs them, so they stay in sync with the code. They run on a CPU in seconds and use a GPU when available.

Random operators with a prescribed contrast. Listing 1 builds 10^4 positive operators in $M_2 \oplus M_3 \oplus M_4$ whose spectra have contrast 0.6 and whose eigenvectors are Haar-random in every sector, and evaluates traces and spectral functions.

Listing 1: Random operators (`ex1_random_operators.py`).

```

"""Random operators with a prescribed spectrum in  $M = M_2 (+) M_3 (+) M_4$ ."""
import torch
from torch_vn_algebra import TypeIAlgebra

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
alg = TypeIAlgebra(n_factors=[2, 3, 4], k_factors=[2, 3, 4], device=dev)
B = 10_000 # Monte Carlo samples, processed as one batch

def contrast_sampler(delta):
    """Spectra in  $[lo, 1]$  with  $\lambda_{\max} = 1$ ,  $\lambda_{\min} = lo$ , i.e. Michelson contrast  $\delta$ ."""
    lo = (1 - delta) / (1 + delta)

    def sampler(dim):
        lam = lo + (1 - lo) * torch.rand(B, dim, device=dev)
        lam[:, 0], lam[:, -1] = 1.0, lo
        return lam
    return sampler

#  $A_c = U_c \text{diag}(\lambda) U_c^*$ , with independent Haar unitaries  $U_c$  in every block and sample
X = alg.operator_from_eigenvalues(contrast_sampler(0.6), batch_size=B,
                                force_positive=True, force_self_adjoint=True)
print(X.matrix.shape) # (10000, 3, 4, 4): batch, sector, padded block
print(X.michelson_contrast[:4]) # 0.6 for every sample
print(X.Tr_blunt().real.mean(), X.Tr_norm().mean(), X.tau_vN().mean())

# functional calculus through the spectral theorem, block by block
R = X.sqrt()
print((R @ R - X).frobenius_norm().max()) # ~1e-6 in single precision
print(X.apply_function(torch.log).tau_vN().mean()) # tau_vN(log X)

```

States, measurement and trace conventions. Listing 2 conditions random states on the outcome of a projective measurement (Lüders rule), which moves all probability into one sector, and checks that expectation values do not depend on the trace used to represent the state.

Listing 2: States and conditioning (`ex2_states_measurement.py`).

```

"""States on  $M$ , sector probabilities, Lueders conditioning and trace conventions."""
import torch
from torch_vn_algebra import TypeIAlgebra, DensityMatrix, states

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
alg = TypeIAlgebra([2, 3], [2, 3], device=dev) # two superselection sectors

rho = states.random_density_matrix(alg, batch_size=4) # Hilbert-Schmidt random states
print(rho.sector_probabilities()) #  $p_c = \text{Tr } \rho_c$ , rows sum to 1

# measure the projection onto the first basis vector of sector 0
P = alg.from_blocks([torch.diag(torch.tensor([1.0, 0.0])), torch.zeros(3, 3)])
post, prob = rho.condition_on(P) #  $\rho|P = P \rho P / \text{Tr}(\rho P)$ 
print(prob, post.sector_probabilities()) # all weight now in sector 0

# the same state as a density w.r.t. tau_vN:  $\omega(A) = \tau_{vN}(d A) = \text{Tr}(\rho A)$ 
A = alg.operator_from_eigenvalues(lambda d: torch.randn(4, d), batch_size=4,
                                force_self_adjoint=True)
d_tau = rho.density('tau_vN')
print(rho.expectation(A).real, (alg.operator(d_tau) @ A).tau_vN().real)

```

```
back = DensityMatrix.from_density(alg, d_tau, trace='tau_vN')
print((back.matrix - rho.matrix).abs().max())
```

A map between sectors. Two fermionic modes have sectors $N = 0, 1, 2$ of dimensions 1, 2, 1. Particle loss with Kraus operators $\sqrt{1-g}N$, $\sqrt{g}a_1$, $\sqrt{g}a_2$ maps sector N to N and $N-1$. Listing 3 builds it from its blocks; the classical part is the transition matrix of a death process,

$$T = \begin{pmatrix} 1 & g & 0 \\ 0 & 1-g & 2g \\ 0 & 0 & 1-2g \end{pmatrix},$$

which the program prints. The last lines illustrate a simple fact: a unital positive map cannot increase the Michelson contrast, since $\lambda_{\min}1 \leq A \leq \lambda_{\max}1$ implies the same bounds for $\Phi(A)$; the non-unital loss channel increases it for most samples.

Listing 3: Particle loss between sectors (ex3_channels.py).

```
"""Particle loss between charge sectors of two fermionic modes, as an InterSectorChannel."""
import torch
from torch_vn_algebra import TypeIAlgebra, InterSectorChannel, channels, states

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
# sectors N = 0, 1, 2 with bases |00>, |10>, |01>; |11>
alg = TypeIAlgebra([1, 2, 1], [1, 2, 1], device=dev)
g = 0.2 # loss probability per particle
s, z = g ** 0.5, torch.zeros
# Kraus operators K_0 = sqrt(1 - g)N, K_1 = sqrt(g) a_1, K_2 = sqrt(g) a_2, split into blocks (d, c)
blocks = {
    (0, 0): [torch.ones(1, 1)],
    (1, 1): [(1 - g) ** 0.5 * torch.eye(2)],
    (2, 2): [(1 - 2 * g) ** 0.5 * torch.ones(1, 1)],
    (0, 1): [z(1, 2), s * torch.tensor([[1.0, 0.0]]), s * torch.tensor([[0.0, 1.0]])],
    (1, 2): [z(2, 1), s * torch.tensor([[0.0], [1.0]]), s * torch.tensor([[1.0], [0.0]])],
}
loss = InterSectorChannel.from_blocks(alg, alg, blocks)
print(loss.is_trace_preserving())
print(loss.transition_matrix()[0]) # classical death process on N

rho = states.random_density_matrix(alg, batch_size=3)
print(loss(rho).sector_probabilities()) # still a DensityMatrix

# unital channels cannot increase the Michelson contrast, non-unital ones can
X = alg.operator_from_eigenvalues(lambda d: 0.5 + torch.rand(1000, d, device=dev),
                                batch_size=1000, force_positive=True)
mix = channels.random_mixed_unitary_channel(alg, 3, batch_size=1000)
print(bool((mix(X).michelson_contrast <= X.michelson_contrast + 1e-6).all()))
print((loss(X).michelson_contrast > X.michelson_contrast).float().mean())
```

Open-system dynamics. Listing 4 integrates spontaneous emission of a two-level atom with the RK4 solver and with the exact exponential of the Lindblad generator, and compares both with $P_e(t) = P_e(0)e^{-\gamma t}$ and $|\rho_{eg}(t)| = |\rho_{eg}(0)|e^{-\gamma t/2}$.

Listing 4: Spontaneous emission (ex4_dynamics.py).

```
"""Spontaneous emission of a two-level atom: Lindblad evolution against the exact solution."""
import math
```

```

import torch
from torch_vn_algebra import TypeIAlgebra, DensityMatrix, dynamics

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
alg = TypeIAlgebra([2], [2], device=dev, precision='double')
omega, gamma = 1.0, 0.3
H = alg.from_blocks([torch.diag(torch.tensor([0.5 * omega, -0.5 * omega]))]) # basis |e>, |g>
L = alg.from_blocks([torch.tensor([[0.0, 0.0], [1.0, 0.0]])]) # sigma_- = |g><e|
psi = torch.tensor([0.6, 0.8], dtype=torch.complex128)
rho0 = DensityMatrix(alg, matrix=torch.outer(psi, psi.conj())[None, None].to(dev))

times = [0.0, 1.0, 2.0, 5.0]
rk4 = dynamics.lindblad_evolve(rho0, H, [L], times, rates=[gamma], substeps=100)
for t, rho in zip(times, rk4):
    exact = dynamics.lindblad_channel(alg, H, [L], t=t, rates=[gamma])(rho0) # exp(tL)
    p_e = 0.36 * math.exp(-gamma * t) # analytic excited population
    coh = 0.48 * math.exp(-gamma * t / 2) # analytic |rho_eg|
    m = rho.matrix[0, 0]
    print(f"t={t}: P_e {m[0, 0].real:.6f} (exact {p_e:.6f}), |rho_eg| {m[0, 1].abs():.6f} "
          f"(exact {coh:.6f}), RK4 vs exp(tL): {(rho.matrix - exact.matrix).abs().max():.1e}")

```

Tensor products. Listing 5 estimates the mean entanglement entropy of random pure states on $\mathbb{C}^m \otimes \mathbb{C}^n$ and compares it with Page’s formula [13].

Listing 5: Page entropy (ex5_page_entropy.py).

```

"""Average entanglement entropy of random pure states on C^m (x) C^n (Page, 1993)."""
import torch
from torch_vn_algebra import TypeIAlgebra, tensor_product, partial_trace, states

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
m, n, B = 4, 8, 20_000
A = TypeIAlgebra([m], [m], device=dev, precision='double')
Bsys = TypeIAlgebra([n], [n], device=dev, precision='double')
AB = tensor_product(A, Bsys)

psi = states.random_density_matrix(AB, batch_size=B, rank=1) # Haar-random pure states
S = states.von_neumann_entropy(partial_trace(psi, keep=1)) # entropy of rho_A

page = sum(1.0 / k for k in range(n + 1, m * n + 1)) - (m - 1) / (2 * n)
print(f"<S_A> = {S.mean():.5f} +/- {S.std() / B ** 0.5:.5f}, Page: {page:.5f}")

```

Searching for a violation. Listing 6 compares Haar sampling with batched gradient ascent for $\sup_U (|\text{Tr}(XUY)| - \text{Tr}(XY))$, whose exact value $\|YX\|_1 - \text{Tr}(XY)$ is positive for non-commuting $X, Y \geq 0$. Section 6 studies this systematically.

Listing 6: Sampling versus optimisation (ex6_inequality_search.py).

```

"""Searching for violations of |Tr(XUY)| <= Tr(XY): random sampling versus gradient ascent."""
import torch
from torch_vn_algebra import TypeIAlgebra

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
torch.manual_seed(0)
k = 16
alg = TypeIAlgebra([k], [k], device=dev, precision='double')
pos = lambda d: 0.2 + torch.rand(1, d)

```

```

X = alg.operator_from_eigenvalues(pos, force_positive=True, force_self_adjoint=True)
Y = alg.operator_from_eigenvalues(pos, force_positive=True, force_self_adjoint=True)
trXY = (X @ Y).trace.real

def z(U):
    # z(U) = |Tr(XUY)| - Tr(XY), batched over U
    return (X @ U @ Y).trace.abs() - trXY

# 1) Monte Carlo over Haar unitaries
print("sampling, 10^5 Haar U:", z(alg.random_unitary_operator(100_000)).max().item())

# 2) gradient ascent over U = U0 exp(G - G*), 64 random starts optimised in parallel;
#     the exponential chart is re-centred at the current point every 50 steps
U0 = alg.random_unitary_operator(64).matrix
for _ in range(10):
    G = torch.zeros(64, 1, k, k, dtype=alg.hilbert.dtype, device=dev, requires_grad=True)
    opt = torch.optim.Adam([G], lr=0.05)
    for step in range(50):
        U = alg.operator(U0) @ alg.operator(G - G.conj().transpose(-2, -1)).expm()
        loss = -z(U).sum()
        opt.zero_grad()
        loss.backward()
        opt.step()
    U0 = U.matrix.detach()
print("gradient ascent:      ", z(alg.operator(U0)).max().item())

# exact supremum: sup_U |Tr(U YX)| = ||YX||_1
print("exact sup:          ", ((Y @ X).trace_norm() - trXY).item())

```

5 Validation against exact results

Table 2 compares Monte Carlo estimates produced with the library (double precision) with exact values: fourth moments of Haar unitaries and orthogonal matrices; the Weingarten-calculus value $\mathbb{E} \tau((AUBU^*)^2) = -1/(N^2 - 1)$ for traceless involutions $A = B$ and Haar U ; Page's formula for $\langle S(\rho_A) \rangle$; and the mean purity $(k + r)/(kr + 1)$ of induced random states of rank r [14]. The last column is the deviation in units of the standard error; all are of order one. For the circular ensembles the Wigner surmise is only an approximation to the exact spacing distribution, so no z -score is given; Figure 1 shows the distributions, with L^1 distances 0.031, 0.021 and 0.029 for $\beta = 1, 2, 4$ from the surmise; level repulsion $P(s) \sim s^\beta$ is clearly visible. The spontaneous-emission example agrees with the analytic solution to 2.2×10^{-16} (exact exponential) and 2.1×10^{-8} (RK4).

6 From sampling to optimisation

The first version of this work [1] studied three trace functionals by Monte Carlo:

$$\begin{aligned}
\text{Exp. 1: } z &= |\text{Tr}(XUY)| - \text{Tr}(XY), & X, Y \geq 0, U \in \mathcal{M} \text{ unitary,} \\
\text{Exp. 2: } z &= \text{Tr}|XY| - \text{Tr}(|X||Y|), & X = UX_0, Y = VY_0, X_0, Y_0 \geq 0, \\
\text{Exp. 3: } z &= \text{Tr}(Y|X|Y) - \text{Tr}|YXY|, & X = X^*, Y \geq 0,
\end{aligned}$$

with $\text{Tr} = \text{Tr}_{\text{blunt}}$, operators normalised to $\lambda_{\max} = 1$, and the contrasts of the operators as parameters. Two observations motivate a different approach.

quantity	parameters	exact	Monte Carlo	std. error	z
$\mathbb{E} U_{11} ^4, U(n)$	$n = 2$	0.333333	0.335015	1.5×10^{-3}	+1.1
$\mathbb{E} U_{11} ^2, U(n)$	$n = 2$	0.5	0.5014	1.4×10^{-3}	+1.0
$\mathbb{E} U_{11} ^4, U(n)$	$n = 4$	0.1	0.0999686	6.8×10^{-4}	-0.0
$\mathbb{E} U_{11} ^4, U(n)$	$n = 8$	0.0277778	0.0277718	2.4×10^{-4}	-0.0
$\mathbb{E} U_{11} ^4, U(n)$	$n = 16$	0.00735294	0.00737778	7.1×10^{-5}	+0.4
$\mathbb{E} U_{11} ^4, U(n)$	$n = 32$	0.00189394	0.00188255	2.0×10^{-5}	-0.6
$\mathbb{E} U_{11} ^2, U(n)$	$n = 32$	0.03125	0.031172	1.5×10^{-4}	-0.5
$\mathbb{E} U_{11} ^4, O(n)$	$n = 2$	0.375	0.373641	1.8×10^{-3}	-0.7
$\mathbb{E} U_{11} ^2, O(n)$	$n = 2$	0.5	0.498956	1.8×10^{-3}	-0.6
$\mathbb{E} U_{11} ^4, O(n)$	$n = 4$	0.125	0.12365	9.8×10^{-4}	-1.4
$\mathbb{E} U_{11} ^4, O(n)$	$n = 8$	0.0375	0.0378452	4.0×10^{-4}	+0.9
$\mathbb{E} U_{11} ^4, O(n)$	$n = 16$	0.0104167	0.0104973	1.4×10^{-4}	+0.6
$\mathbb{E} U_{11} ^4, O(n)$	$n = 32$	0.00275735	0.0027189	3.9×10^{-5}	-1.0
$\mathbb{E} U_{11} ^2, O(n)$	$n = 32$	0.03125	0.0311461	2.1×10^{-4}	-0.5
$\mathbb{E} \tau((AUBU^*)^2)$	$N = 4$	-0.0666667	-0.0668132	1.8×10^{-3}	-0.1
$\mathbb{E} \tau((AUBU^*)^2)$	$N = 16$	-0.00392157	-0.00342506	4.0×10^{-4}	+1.3
$\mathbb{E} \tau((AUBU^*)^2)$	$N = 64$	-0.0002442	-0.000192229	2.0×10^{-4}	+0.3
$\langle s^2 \rangle, \text{COE}$	$N = 32$	1.27324	1.28476	5.2×10^{-3}	-
$\langle s^2 \rangle, \text{CUE}$	$N = 32$	1.1781	1.17783	3.8×10^{-3}	-
$\langle s^2 \rangle, \text{CSE}$	$N = 32$	1.10447	1.10321	3.9×10^{-3}	-
$\langle S(\rho_A) \rangle$	$m = 2, n = 8$	0.600372	0.600235	9.6×10^{-4}	-0.1
$\langle S(\rho_A) \rangle$	$m = 4, n = 8$	1.15314	1.15389	1.0×10^{-3}	+0.7
$\langle S(\rho_A) \rangle$	$m = 8, n = 8$	1.58853	1.58838	8.7×10^{-4}	-0.2
$\mathbb{E} \text{Tr} \rho^2$	$k = 4, r = 2$	0.666667	0.666228	1.0×10^{-3}	-0.4
$\mathbb{E} \text{Tr} \rho^2$	$k = 4, r = 4$	0.470588	0.470432	6.8×10^{-4}	-0.2
$\mathbb{E} \text{Tr} \rho^2$	$k = 16, r = 4$	0.307692	0.307736	2.0×10^{-4}	+0.2
$\mathbb{E} \text{Tr} \rho^2$	$k = 16, r = 16$	0.124514	0.124468	5.4×10^{-5}	-0.8

Table 2: Monte Carlo estimates versus exact values (quick run on CPU (4 threads), PyTorch 2.14.1+cu130).

Sampling misses the extremes. For fixed X, Y the supremum in Exp. 1 is known exactly, $\sup_U z = \|YX\|_1 - \text{Tr}(XY) > 0$. Table 3 and Figure 2 show that the best of 20 000 Haar samples is already negative for $k \geq 4$ and moves further away from the supremum as k grows, while batched gradient ascent (32 starts, 500 steps) finds it to a relative accuracy of 10^{-5} or better. This is concentration of measure: the unitaries close to the maximiser form a set whose Haar measure decays rapidly with the dimension. In particular, the absence of violations in Monte Carlo samples for $k = 16$ reported in [1] is a property of the sampling, not of the inequality.

Constrained optimisation. The natural objects are the envelopes

$$\bar{z}(\delta) = \sup\{z : \Delta(X) = \delta\}, \quad \underline{z}(\delta) = \inf\{z : \Delta(X) = \delta\},$$

(and the same with $\Delta(Y)$, or with both contrasts prescribed), the supremum being taken over all remaining variables. The module `optimize` parametrises the constraint set exactly: the spectrum is $\lambda_{\max} = 1$, $\lambda_{\min} = (1 - \delta)/(1 + \delta)$ and the other eigenvalues in between (through a sigmoid), the eigenvectors are $U = U_0 e^{G - G^*}$ with a Haar-random U_0 per start and a chart that is re-centred after every round, and for Exp. 3 the signs of the eigenvalues of X are drawn per start and kept fixed. Every grid point and every start is one element of the batch, so the whole envelope is computed in a single batched optimisation.

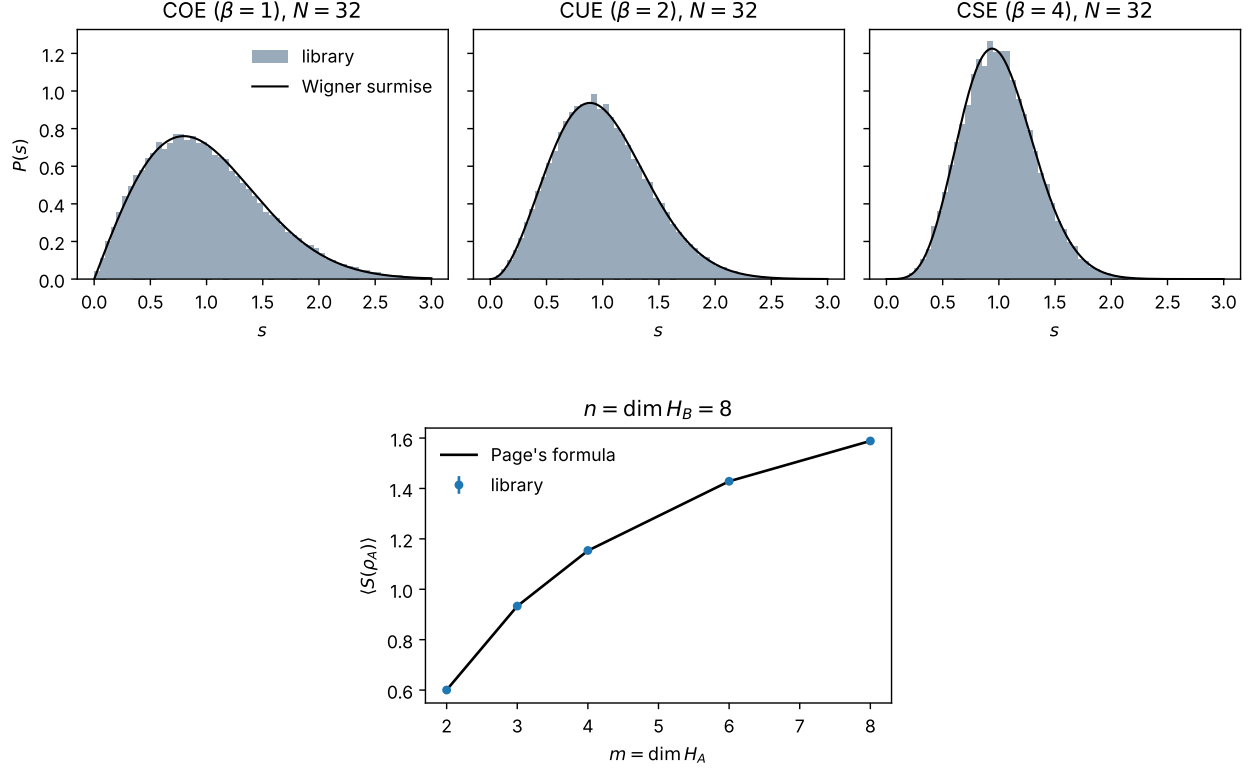


Figure 1: Top: nearest-neighbour spacings of COE, CUE and CSE eigenphases generated by the library, against the Wigner surmise. Bottom: mean entanglement entropy of random pure states against Page's formula (error bars: two standard errors).

Figures 3 and 4 and Table 4 show the result. Every Monte Carlo sample of [1] lies between the computed envelopes; samples that fall outside the linear interpolation between grid points are re-optimised at their exact contrast, after which none remains outside. The envelopes are much wider than the sample clouds, already for $k = 2$.

Two envelopes can be checked analytically. In Exp. 3, $z \geq 0$ always:

Proposition 1. *For $X = X^*$ and $Y \geq 0$, $\text{Tr} |YXY| \leq \text{Tr}(Y|X|Y)$, with equality if Y commutes with X .*

Proof. With $X = X_+ - X_-$, $|X| = X_+ + X_-$, the triangle inequality for the trace norm gives $\|YX_+Y - YX_-Y\|_1 \leq \text{Tr}(YX_+Y) + \text{Tr}(YX_-Y)$. If $XY = YX$, then $|YXY| = Y^2|X|$. $\square$

The computed lower envelope of Exp. 3 is zero to machine precision. In Exp. 1, the lower envelope for prescribed $\Delta(X) = \delta$ equals $-(kC - 2\delta/(1 + \delta))$ when all sectors have size $k \geq 2$: $\text{Tr}(XY) \leq \text{Tr} X \leq kC - 1 + \lambda_{\min}(X)$ with equality for $Y = 1$, while $|\text{Tr}(UYX)|$ can be made zero by a cyclic permutation of the eigenbasis of X_c in every sector. The optimiser reproduces this curve to a relative deviation of $8.1\text{e-}03$. The upper envelopes of all three experiments are, to our knowledge, not known in closed form; the computed curves are lower bounds for them (a local search) and suggest conjectures for sharp contrast-dependent inequalities, which we leave for future work.

k	violation found		$\sup z / \text{Tr } XY$	best sample $z / \text{Tr } XY$	ascent rel. error		time [s]	
	sampling	ascent			median	max	sampl.	ascent
2	2/4	4/4	4.4×10^{-5}	-0.00	5.4×10^{-8}	1.7×10^{-5}	0.015	0.86
4	0/4	4/4	2.0×10^{-3}	-0.19	9.4×10^{-8}	7.5×10^{-7}	0.049	0.86
8	0/4	4/4	3.2×10^{-3}	-0.58	5.0×10^{-7}	2.1×10^{-6}	0.18	1.2
16	0/4	4/4	4.5×10^{-3}	-0.78	2.6×10^{-6}	4.4×10^{-6}	0.77	2.6

Table 3: Exp. 1 for fixed random X, Y : Haar sampling versus gradient ascent (medians over 4 pairs; quick run on CPU (4 threads)).

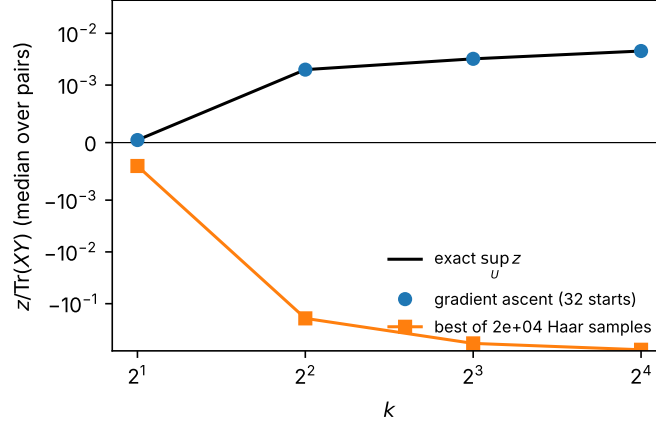


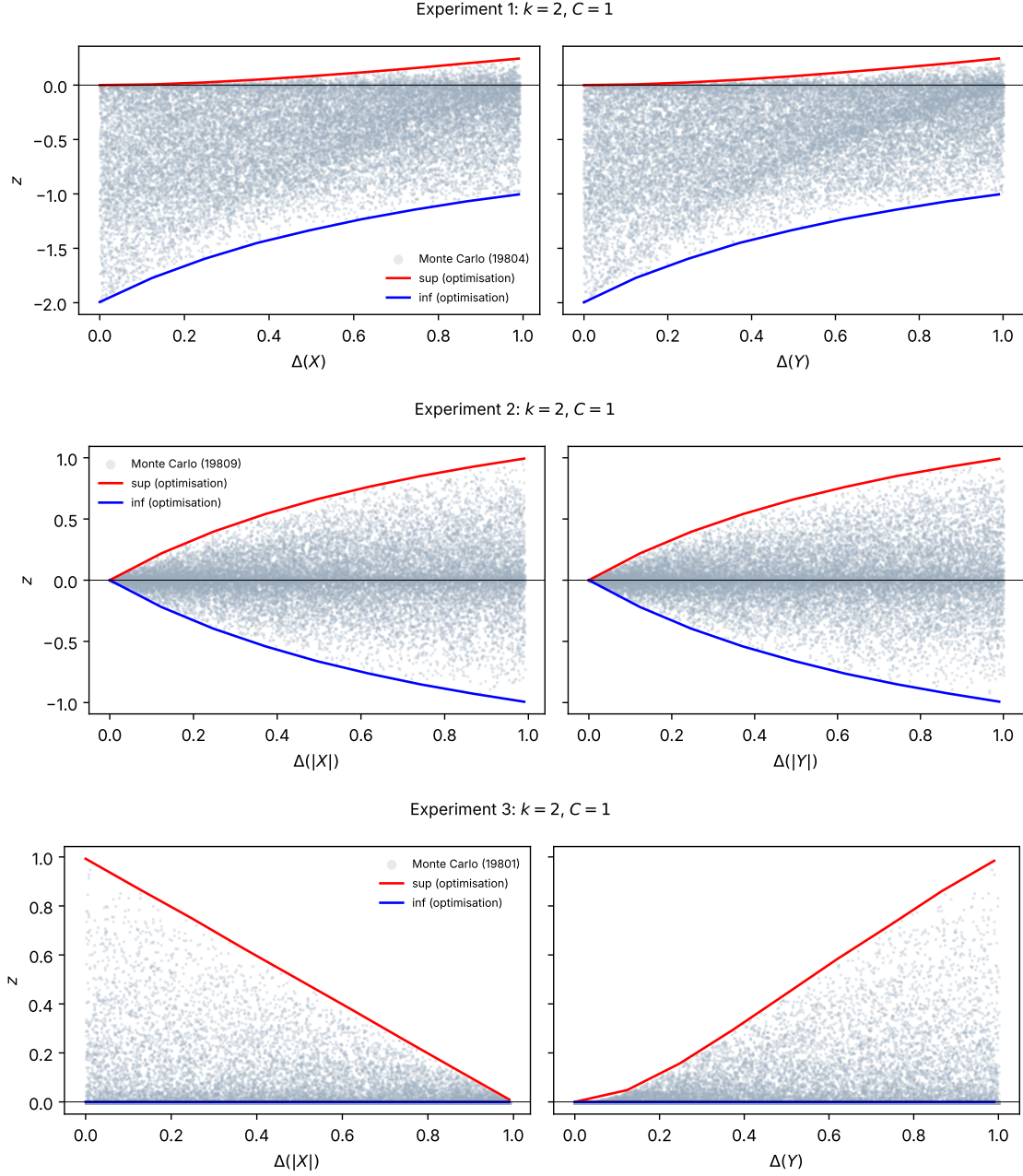
Figure 2: Best z found by sampling and by gradient ascent, relative to $\text{Tr}(XY)$, against the exact supremum (symmetric log scale).

7 Performance

Table 5 and Figure 5 report the wall-clock time per sample of library operations (not of isolated kernels), in single precision, with the batch size chosen so that the batch holds about 2^{22} matrix entries (quick run on CPU (4 threads); CPU with 4 threads). Expected behaviour: for single small blocks the CPU is faster, because kernel launches dominate; the GPU gains for many sectors and for the matrix products in channels and in the optimisation step; batched symmetric eigen- and singular-value solvers lose much of their advantage from $k \approx 64$ on, where the GPU libraries switch from batched small-matrix kernels to general ones.

8 Limitations and roadmap

- *Layout.* Blocks are padded to $k_{\max}$; for very different sector sizes (e.g. binomial sectors of spin chains) a bucketed layout is needed. Merging sectors of equal total charge in tensor products (fusion rules) is not implemented yet.
- *Optimisation.* The search is local; envelopes are lower bounds for suprema (upper bounds for infima). Gradients of spectral functions are singular at degenerate spectra; the parametrisations avoid this for the contrast constraint, but not for every user-defined objective.
- *Scale.* Choi matrices, superoperators and $e^{t\mathcal{L}}$ work with $k_c^2 \times k_c^2$ matrices and are limited to blocks of a few tens; RK4 integration scales to larger blocks.



Exp.	k	C	$\inf z$	$\sup z$	Monte Carlo range of z	outside
1	2	1	-1.994	+0.2436	$[-1.98, +0.225]$	0
1	2	2	-3.991	+0.4765	$[-3.39, +0.328]$	0
1	4	1	-3.994	+0.472	—	—
2	2	1	-0.9931	+0.9934	$[-0.921, +0.975]$	0
2	2	2	-1.956	+1.963	$[-1.43, +1.72]$	0
2	4	1	-1.961	+1.957	—	—
3	2	1	0	+0.9927	$[0, +0.946]$	0
3	2	2	0	+1.968	$[0, +1.39]$	0
3	4	1	0	+1.962	—	—

Table 4: Global extremes of z over all contrasts found by constrained optimisation, the range of the Monte Carlo samples, and the number of samples outside the envelopes after re-optimisation (12 starts per grid point, 240 steps; quick run on CPU (4 threads)).

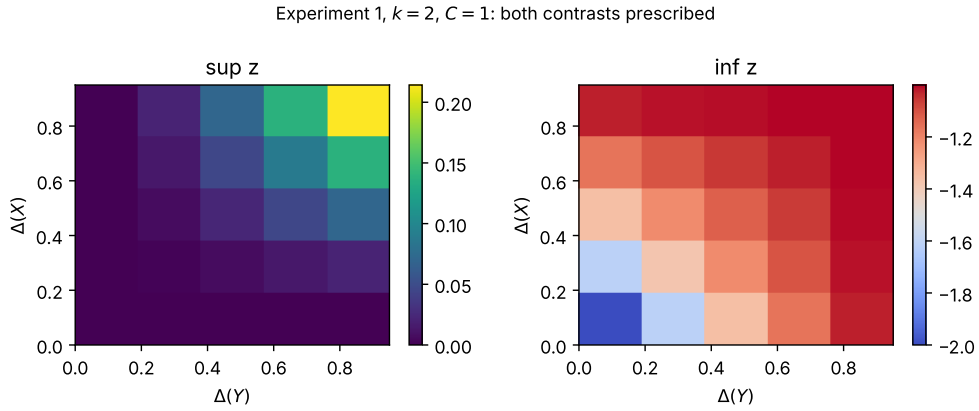


Figure 4: Exp. 1, $k = 2$, $C = 1$: $\sup z$ and $\inf z$ with both contrasts prescribed.

- *Scope.* Finite dimensions and Type I only; no tensor-network or sparse methods.

9 Availability and reproducibility

The code (MIT licence) is available at https://github.com/andre-a-no/von_neumann_type_i. `bash scripts/paper/run_all.sh` full cuda regenerates every table and figure of this text on a GPU; `bash scripts/paper/run_all.sh` runs a reduced version on a CPU in a few minutes. The tables state on which device and in which mode they were produced.

References

- [1] I. Nikolaeva, A. Novikov, “Finite-dimensional Type I von Neumann algebras in PyTorch: a GPU-accelerated framework for random block-diagonal operators”, arXiv:2606.15882v1 (2026).
- [2] L. T. Gardner, “An inequality characterizes the trace”, *Canad. J. Math.* 31 (1979), 1322–1328.
- [3] D. Petz, J. Zemánek, “Characterizations of the trace”, *Linear Algebra Appl.* 111 (1988), 43–52.
- [4] A. Novikov, O. Tikhonov, “Characterization of central elements of operator algebras by inequalities”, arXiv:1502.01267 (2015).

operation	k	C	batch	time per sample [μ s]
generate	4	1	4096	1.33
f(A) eigh	4	1	4096	3.19
A SVD	4	1	4096	6.14
contrast	4	1	4096	2.07
channel	4	1	4096	0.595
exp(tL)	4	1	4096	59.6
opt. step	4	1	4096	13.6
generate	4	16	1024	28.1
f(A) eigh	4	16	1024	44.1
A SVD	4	16	1024	90
contrast	4	16	1024	24.3
channel	4	16	1024	7.81
exp(tL)	4	16	1024	1.42e+03
opt. step	4	16	1024	314
generate	16	1	1024	18.1
f(A) eigh	16	1	1024	35.8
A SVD	16	1	1024	67.2
contrast	16	1	1024	14
channel	16	1	1024	2.96
exp(tL)	16	1	1024	1.67e+04
opt. step	16	1	1024	205
generate	16	16	64	439
f(A) eigh	16	16	64	495
A SVD	16	16	64	938
contrast	16	16	64	200
channel	16	16	64	46.7
exp(tL)	16	16	64	2.48e+05
opt. step	16	16	64	3e+03

Table 5: Time per sample of library operations.

- [5] D. Virosztek, “Connections between centrality and local monotonicity of certain functions”, J. Math. Anal. Appl. 453 (2017), 221–226.
- [6] S. A. Abed, I. A. Nikolaeva, A. A. Novikov, “Generalisation of Michelson contrast for operators and its properties”, Lobachevskii J. Math. 45 (2024), 3835–3848.
- [7] A. Novikov, S. A. Abed, I. Nikolaeva, “Generalisation of Michelson contrast for operators”, arXiv:2010.10130 (2020).
- [8] J. R. Johansson, P. D. Nation, F. Nori, “QuTiP 2: A Python framework for the dynamics of open quantum systems”, Comput. Phys. Commun. 184 (2013), 1234–1240.
- [9] Qiskit contributors, “Qiskit: An open-source framework for quantum computing”, 2019, <https://qiskit.org>.
- [10] P. Guilmin, R. Gautier, A. Bocquet, E. Genois, D. Weiss, “dynamics: an open-source Python library for GPU-accelerated and differentiable simulation of quantum systems”, <https://github.com/dynamics/dynamics>.
- [11] F. Mezzadri, “How to generate random matrices from the classical compact groups”, Notices Amer. Math. Soc. 54 (2007), 592–604.

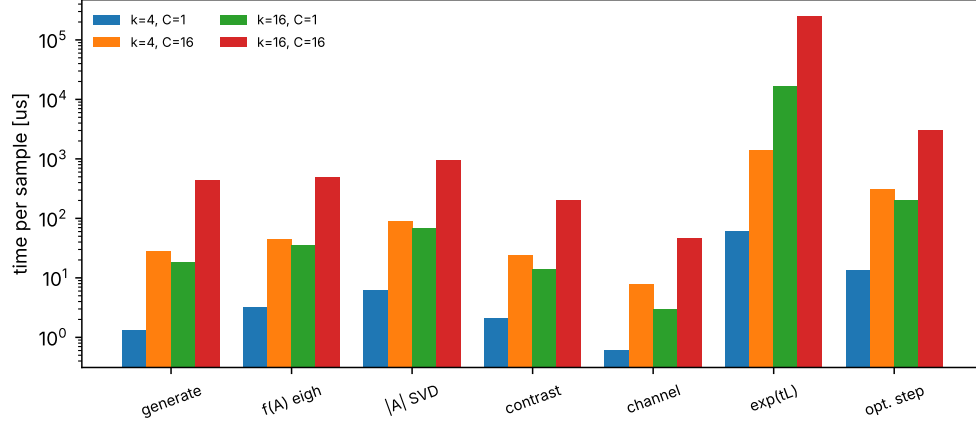


Figure 5: Time per sample of library operations.

- [12] M. L. Mehta, *Random Matrices*, 3rd ed., Elsevier, 2004.
- [13] D. N. Page, “Average entropy of a subsystem”, *Phys. Rev. Lett.* 71 (1993), 1291–1294.
- [14] K. Życzkowski, H.-J. Sommers, “Induced measures in the space of mixed quantum states”, *J. Phys. A* 34 (2001), 7111–7125.
- [15] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library”, *NeurIPS* 2019.